

Jamere Golden

Homework 3 Questions And Answers

- 1. You are asked to design a switching network to connect 50 inputs to 50 outputs. You want to achieve a maximum utilization of 40%. Inputs are organized in 5 groups and so are outputs. Draw the corresponding diagram with all relevant connections. Compute the total number of crosspoints.***

Explanation of Design:

Input Stage:

There are 5 group switches, each handling 10 inputs.
Each switch connects its group of inputs to k middle stage switches.

Middle Stage:

We have a set of intermediate switches that connect the input stage to the output stage.
Each middle stage switch is fully connected to each input group switch and output group switch.
To achieve 40% utilization, we use redundancy by increasing the number of middle stage switches.

Output Stage:

There are 5 group switches, each handling 10 outputs.
Each middle stage switch connects to all output group switches.

Calculation of Crosspoints:

Input Stage:

Each group switch handles 10 inputs.
Assume each input group switch connects to k middle stage switches.
If we use $k = 8$ (to ensure redundancy and achieve 40% utilization), each input switch needs $10 \times 8 = 80$ crosspoints.

There are 5 input switches, so total crosspoints in the input stage:
 $5 \times 80 = 400$

Middle Stage:

There are 8 middle stage switches, each connecting to all 5 input groups and all 5 output groups.

Each middle switch connects to $10 \times 2 = 20$ crosspoints (10 from inputs, 10 to outputs) for each group.

Thus, each middle switch has $20 \times 5 = 100$ crosspoints.

Total crosspoints in the middle stage: $8 \times 100 = 800$.

Output Stage:

Each output group switch connects to 8 middle stage switches with 10 outputs.

Each output switch has $10 \times 8 = 80$ crosspoints.

There are 5 output switches, so total crosspoints in the output stage: $5 \times 80 = 400$.

Total Crosspoints:

Input Stage: 400

Middle Stage: 800

Output Stage: 400

Total = $400 + 800 + 400 = 1,600$ crosspoints

This multistage design is efficient for handling the connections while keeping utilization under control.

2. Analyze in detail both an ARP request and an ARP reply packet using Wireshark. Submit relevant snapshots.

1. ARP Request Packet Analysis

The ARP request packet asks, "Who has IP address X.X.X.X?" Here's how to analyze it:

Expand the ARP section in the packet details.

Key fields to examine:

Hardware type: 1 (Ethernet)

Protocol type: 0x0800 (IPv4)

Hardware size: 6 bytes (MAC address size)

Protocol size: 4 bytes (IPv4 address size)

Opcode: 1 (ARP Request)

Sender MAC address: MAC address of the sender device

Sender IP address: IP address of the sender device

Target MAC address: 00:00:00:00:00:00 (unknown)

Target IP address: The IP address being queried

2. ARP Reply Packet Analysis

The ARP reply packet responds, "I am at IP address X.X.X.X." Here's how to analyze it:

Expand the ARP section in the packet details.

Key fields to examine:

Hardware type: 1 (Ethernet)

Protocol type: 0x0800 (IPv4)

Hardware size: 6 bytes

Protocol size: 4 bytes

Opcode: 2 (ARP Reply)

Sender MAC address: MAC address of the responder

Sender IP address: IP address of the responder (the queried IP)

Target MAC address: MAC address of the original requester

Target IP address: IP address of the original request

Wireshark capture of ARP requests. The packet list shows several ARP requests from various sources to the destination 00:08:0d:6e:5e:17:d5. The packet details pane shows the structure of the Ethernet II frame and the ARP request.

No.	Time	Source	Destination	Protocol	Length	Info
20222	747.536240	20:88:10:99:3c:d2	Broadcast	ARP	60	who has 10.4.2.145? Tell 10.4.0.38
20225	747.580286	20:88:10:99:3b:6a	Broadcast	ARP	60	who has 10.4.6.140? Tell 10.4.6.16
20230	747.792683	90:8d:6e:60:79:ed	Broadcast	ARP	60	who has 10.4.0.71? Tell 10.4.1.115
20241	747.902489	20:88:10:99:37:2d	Broadcast	ARP	60	who has 10.4.7.226? Tell 10.4.2.68
20242	747.926050	90:8d:6e:5e:9b:20	Broadcast	ARP	60	who has 10.4.3.200? Tell 10.4.3.79
20247	748.105659	90:8d:6e:5e:a8:ec	Broadcast	ARP	60	who has 10.4.2.230? Tell 10.4.1.90
20262	748.580351	20:88:10:99:3b:6a	Broadcast	ARP	60	who has 10.4.6.140? Tell 10.4.6.16
20263	748.643379	90:8d:6e:5e:17:d5	Broadcast	ARP	60	who has 10.4.3.200? Tell 10.4.7.170
20277	749.103558	20:88:10:99:3d:00	Cisco_7f:fa:00	ARP	42	who has 10.4.1.1? Tell 10.4.5.107
20278	749.104234	Cisco_7f:fa:00	20:88:10:99:3d:00	ARP	60	10.4.1.1 is at 00:78:88:7f:fa:00
20279	749.106355	90:8d:6e:5e:a8:ec	Broadcast	ARP	60	who has 10.4.2.230? Tell 10.4.1.90
20284	749.198074	20:88:10:99:36:ee	Broadcast	ARP	60	who has 10.4.0.113? Tell 10.4.0.177
20287	749.217465	Cisco_7f:fa:00	Broadcast	ARP	60	who has 10.4.2.83? Tell 10.4.1.1
20291	749.258873	90:8d:6e:60:ac:22	Broadcast	ARP	60	who has 10.4.2.230? Tell 10.4.3.242
20293	749.308105	90:8d:6e:5e:17:d5	Broadcast	ARP	60	who has 10.4.3.200? Tell 10.4.7.170

Frame 20277: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
 Ethernet II, Src: 20:88:10:99:3d:00 (20:88:10:99:3d:00), Dst: Cisco_7f:fa:00 (00:78:88:7f:fa:00)
 Address Resolution Protocol (request)

Wireshark capture of ARP replies. The packet list shows several ARP replies from various sources to the destination 00:08:0d:6e:5e:17:d5. The packet details pane shows the structure of the Ethernet II frame and the ARP reply.

No.	Time	Source	Destination	Protocol	Length	Info
20222	747.536240	20:88:10:99:3c:d2	Broadcast	ARP	60	who has 10.4.2.145? Tell 10.4.0.38
20225	747.580286	20:88:10:99:3b:6a	Broadcast	ARP	60	who has 10.4.6.140? Tell 10.4.6.16
20230	747.792683	90:8d:6e:60:79:ed	Broadcast	ARP	60	who has 10.4.0.71? Tell 10.4.1.115
20241	747.902489	20:88:10:99:37:2d	Broadcast	ARP	60	who has 10.4.7.226? Tell 10.4.2.68
20242	747.926050	90:8d:6e:5e:9b:20	Broadcast	ARP	60	who has 10.4.3.200? Tell 10.4.3.79
20247	748.105659	90:8d:6e:5e:a8:ec	Broadcast	ARP	60	who has 10.4.2.230? Tell 10.4.1.90
20262	748.580351	20:88:10:99:3b:6a	Broadcast	ARP	60	who has 10.4.6.140? Tell 10.4.6.16
20263	748.643379	90:8d:6e:5e:17:d5	Broadcast	ARP	60	who has 10.4.3.200? Tell 10.4.7.170
20277	749.103558	20:88:10:99:3d:00	Cisco_7f:fa:00	ARP	42	who has 10.4.1.1? Tell 10.4.5.107
20278	749.104234	Cisco_7f:fa:00	20:88:10:99:3d:00	ARP	60	10.4.1.1 is at 00:78:88:7f:fa:00
20279	749.106355	90:8d:6e:5e:a8:ec	Broadcast	ARP	60	who has 10.4.2.230? Tell 10.4.1.90
20284	749.198074	20:88:10:99:36:ee	Broadcast	ARP	60	who has 10.4.0.113? Tell 10.4.0.177
20287	749.217465	Cisco_7f:fa:00	Broadcast	ARP	60	who has 10.4.2.83? Tell 10.4.1.1
20291	749.258873	90:8d:6e:60:ac:22	Broadcast	ARP	60	who has 10.4.2.230? Tell 10.4.3.242
20293	749.308105	90:8d:6e:5e:17:d5	Broadcast	ARP	60	who has 10.4.3.200? Tell 10.4.7.170

Frame 20278: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
 Ethernet II, Src: Cisco_7f:fa:00 (00:78:88:7f:fa:00), Dst: 20:88:10:99:3d:00 (20:88:10:99:3d:00)
 Address Resolution Protocol (reply)

3. Given dataword $d = 110011010$ and the divisor 11011 , show the generation of the CRC codeword at the sender site (using binary division)

Step-by-Step Binary Division for CRC

Inputs:

Dataword: 110011010

Divisor: 11011 (5 bits long)

Appended Dataword: 1100110100000 (4 zeros appended, as the divisor degree is 4).

Step 1: Initialize Division

Start with the first 5 bits from the padded dataword (11001) and XOR with the divisor:

$$11001 / 11011 = 00010$$

Bring down the next bit (1), forming the new dividend:

000101

Step 2: Process Next Step

Since the first bit of 000101 is 0, no XOR is performed with the divisor. Bring down the next bit (0):

0001010

Step 3: Continue Division

Again, since the first bit is (0), no XOR is performed. Bring down the next bit (1):

00010101

Now perform XOR with the divisor (11011):

$$10101 / 11011 = 01110$$

Bring down the next bit (0), forming the new dividend:

011100

Step 4: Continue Division

Perform XOR with the divisor again:

$$11100 / 11011 = 00111$$

Bring down the next bit (0), forming the new dividend:

001110

Step 5: Final Steps

Perform XOR with the divisor again:

$$1110 / 11011 = 0111$$

Since there are no more bits to bring down, the remainder is 0111.

Result

Remainder CRC: 0111

CRC Codeword: Original dataword 110011010 appended with the remainder 0111:

Codeword: 1100110100111

- 4. Given the following set of bytes (represented as decimal numbers), produce the checksum in its negated form: [89, 76, 133, 244, 205, 162, 139, 37, 209, 52]. Show that a correct transmission yields a checksum of 0 at the recipient's side (represented in 1's complement notation).**

Given Bytes:

[89, 76, 133, 244, 205, 162, 139, 37, 209, 52]

Step 1: Sum the Bytes

We sum the bytes step by step:

$$89 + 76 = 165$$

$$165 + 133 = 298$$

$$298 + 244 = 542$$

$$542 + 205 = 747$$

$$747 + 162 = 909$$

$$909 + 139 = 1048$$

$$1048 + 37 = 1085$$

$$1085 + 209 = 1294$$

$$1294 + 52 = 1346$$

So, the sum of the bytes is 1346.

Step 2: Applying 16-bit Arithmetic (Modulo 65536)

Since the sum is 1346, which is well below the 16-bit maximum value (65535), there is no need to carry over. Thus, the sum remains 1346.

Step 3: Negating the Checksum (1's Complement)

Now, we negate the checksum in 1's complement by subtracting the sum from 65535:

$$65535 - 1346 = 64189$$

Thus, the checksum in 1's complement is 64189.

Step 4: Verification

For a correct transmission, the sum of the data (1346) and the checksum (64189) should add up to 65535 in 1's complement:

$$1346 + 64189 = 65535$$

This confirms that the checksum works correctly.

Conclusion:

- The correct checksum in its negated form (1's complement) is 64189.
- A correct transmission will yield a checksum of 0 in 1's complement.